

SISTEM MANAJEMEN DOKUMEN BERBASIS COMMAND LINE INTERFACE MENGGUNAKAN BAHASA PEMOGRAMAN PYTHON



Disusun untuk memenuhi tugas akhir Project-Based Learning (PBL)

Mata Kuliah Algoritma & Struktur Data (TPL2106)

Disusun oleh:

Kelompok 8

Cinta Asih Salaamina - J0403251045

Najwa Hanifa Sholiha Asy Syifa SM - J0403251113

Selena Rohmani Putri - J0403251131

Program Studi Teknologi Rekayasa Perangkat Lunak

Sekolah Vokasi IPB University

2026

KATA PENGANTAR

Segala puji dan rasa syukur senantiasa penulis haturkan kepada Tuhan Yang Maha Esa atas kelancaran dan petunjuk-Nya, sehingga laporan akhir *Project-Based Learning* (PBL) yang berjudul "Sistem Manajemen Dokumen Berbasis Command Line Interface Menggunakan Bahasa Pemograman Python" ini dapat dirampungkan tepat pada waktunya. Penyusunan laporan ini merupakan wujud dari penerapan praktis ilmu yang didapatkan guna memenuhi tugas akhir mata kuliah Algoritma dan Struktur Data pada Program Studi Teknologi Rekayasa Perangkat Lunak, Sekolah Vokasi IPB University.

Penulis menyadari bahwa proses pengerjaan proyek ini tidak lepas dari dukungan berbagai pihak. Oleh karena itu, ungkapan terima kasih yang sebesar-besarnya ditujukan kepada Ibu Dr. Shelve Nidya Neyman S.Kom., M.Si. dan Ibu Dr. Sofiyanti Indriasari S.Kom., M.Kom., selaku dosen pengampu mata kuliah Algoritma dan Struktur Data. Bimbingan, wawasan, serta motivasi yang beliau berikan sangat berharga bagi penulis dalam memecahkan masalah komputasional serta memahami esensi struktur data dalam kasus dunia nyata.

Proyek ini mengangkat studi kasus pengelolaan tumpukan dokumen meja kerja yang diimplementasikan menggunakan prinsip LIFO (*Last In First Out*) melalui struktur data Stack, dibantu dengan antarmuka grafis terminal berbasis *Command Line Interface* (CLI). Penulis menyadari bahwa hasil akhir dari aplikasi maupun penulisan laporan komprehensif ini masih memiliki ruang untuk penyempurnaan. Segala masukan dan kritik konstruktif sangat dinantikan agar karya-karya selanjutnya dapat menjadi lebih baik dan solutif. Harapan penulis, semoga laporan ini mampu memberikan perspektif baru dan manfaat nyata bagi para pembacanya, khususnya dalam penerapan struktur data komputasional pada tata kelola administrasi modern.

Bogor, 24 Juni 2026

Kelompok 8

DAFTAR ISI

KATA PENGANTAR	2
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang.....	5
1.2 Tujuan Proyek.....	5
1.3 Ruang Lingkup	6
BAB II ANALISIS DAN PERANCANGAN	6
2.1 Deskripsi Sistem	6
2.2 Struktur Data yang Digunakan	7
2.3 Algoritma yang Digunakan	8
2.4 Diagram Alir Program (Flowchart).....	9
2.5 Struktur Program	9
BAB III IMPLEMENTASI PROGRAM	10
3.1 Tampilan Menu Utama.....	10
3.2 Fitur Create.....	11
3.3 Fitur Read.....	11
3.4 Fitur Update.....	12
3.5 Fitur Delete.....	13
3.6 Fitur Searching	14
3.7 Fitur Sorting.....	14
3.8 Fitur Proses Tumpukan	15
3.9 Penyimpanan Data (File Handling)	15
3.10 Validasi Input dan Error Handling	16
BAB IV PENGUJIAN DAN ANALISIS	17
4.1 Skenario Pengujian	17
4.2 Kelebihan Sistem	18
4.3 Keterbatasan Sistem.....	18
BAB V KENDALA DAN SOLUSI	19
BAB VI PEMBAGIAN TUGAS ANGGOTA	20

BAB VII KESIMPULAN DAN SARAN	21
DAFTAR PUSTAKA	23
LAMPIRAN	24
Lampiran A. Link GitHub	24
Lampiran B. Screenshot Program.....	24
Lampiran C. Struktur Folder Proyek	28
Lampiran D. Commit History GitHub	28
Lampiran E. Source Code Utama	30

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam lingkungan kerja profesional, penumpukan dokumen di atas meja merupakan masalah klasik yang sering kali menghambat produktivitas. Proses administrasi dan pengelolaan surat atau berkas yang masih mengandalkan pencatatan manual sering menyebabkan berkas penting terselip atau lambat untuk diproses. Kondisi ini menuntut adanya sistem pengarsipan yang efisien, di mana urutan pemrosesan dokumen diatur dengan logika yang sistematis.

Salah satu pendekatan natural yang terjadi di meja kerja fisik adalah dokumen atau surat yang baru saja masuk akan diletakkan di tumpukan teratas, dan secara logis, dokumen teratas tersebutlah yang akan diproses terlebih dahulu. Meniru pola kerja fisik tersebut, sistem digital dapat diimplementasikan menggunakan konsep LIFO (*Last In First Out*) yang merupakan karakteristik utama dari struktur data *Stack*.

Berdasarkan permasalahan di atas, proyek ini mengembangkan aplikasi bernama "Sistem Manajemen Dokumen Berbasis Command Line Interface Menggunakan Bahasa Pemrograman Python". Proyek ini dibuat untuk membantu pengelolaan antrean tumpukan dokumen secara lebih terstruktur. Sistem mampu menyimpan data secara permanen menggunakan file serta mengimplementasikan struktur data *Stack* untuk mengatur antrean surat masuk agar diproses sesuai kaidah prioritas dokumen terbaru.

1.2 Tujuan Proyek

1. Membuat aplikasi berbasis terminal (*Command Line Interface*) menggunakan bahasa pemrograman Python.
2. Mengimplementasikan struktur data *Stack* (LIFO) dan algoritma yang telah dipelajari untuk menyelesaikan permasalahan pengurutan prioritas.
3. Menerapkan konsep *file handling* agar data arsip dan tumpukan tersimpan secara permanen di repositori lokal.
4. Mengembangkan solusi terhadap permasalahan nyata mengenai tumpukan administrasi perkantoran agar lebih tertata dan terdigitalisasi.

1.3 Ruang Lingkup

- Aplikasi berjalan di lingkungan terminal (CLI) menggunakan kerangka kerja antarmuka TUI (*Text User Interface*) dari pustaka Python "*Textual*".
- Bahasa pemrograman Python
- Penyimpanan data memanfaatkan metode *file handling* konvensional berbasis `.txt` (`data.txt` dan `riwayat.txt`).
- Fitur aplikasi meliputi operasi dasar manajemen data (*Create, Read, Update, Delete*) pada catatan dokumen.
- Fitur Searching menggunakan algoritma *Sequential Search*.
- Fitur Sorting menggunakan algoritma *Merge Sort*.
- Fungsionalitas logika inti sistem berfokus pada fitur "Proses (LIFO)" yang merepresentasikan pelepasan (pop) dari tumpukan teratas.
- Aplikasi dapat mengakomodir salinan file fisik berupa `.pdf` dan `.docx` ke dalam direktori khusus (`file_dokumen/`).

BAB II ANALISIS DAN PERANCANGAN

2.1 Deskripsi Sistem

Sistem Manajemen Dokumen ini dirancang sebagai solusi perantara (middleware) antara pengelolaan berkas fisik dan digitalisasi arsip murni. Sistem secara konseptual memodelkan "Meja Kerja" administrator di mana setiap surat fisik yang diterima dicatat meta-datanya ke dalam aplikasi, sementara berkas lunaknya (*soft copy*) diunggah sebagai lampiran. Model antrean yang digunakan secara eksklusif adalah tumpukan (Stack), yang memaksa proses penyelesaian dilakukan dari dokumen paling baru yang masuk ke sistem (dokumen di posisi teratas).

Paradigma interaksi yang dipilih adalah antarmuka baris perintah (CLI) namun diperkaya dengan elemen *Text User Interface* (TUI). Pemilihan TUI dibandingkan GUI tradisional (seperti Tkinter atau PyQt) didasarkan pada kebutuhan pengoperasian yang cepat di lingkungan peladen (server) atau lingkungan terminal perkantoran yang mengutamakan navigasi papan ketik (keyboard-driven) tanpa

menghilangkan aspek visual yang interaktif seperti tombol, kolom masukan (input fields), dan panel.

2.2 Struktur Data yang Digunakan

Struktur Data	Fungsi
Linked List/List (Sebagai Stack)	Menyimpan data utama tumpukan dokumen. List di Python dioperasikan menggunakan prinsip Stack murni (menggunakan <code>append()</code> untuk menambah elemen di akhir array dan <code>pop()</code> untuk mengambil/menghapus elemen di paling akhir).
Dictionary	Mengelola dan menyimpan rincian informasi satu entitas dokumen secara terstruktur. Setiap dokumen direpresentasikan sebagai pasangan <i>key-value</i> yang memuat atribut seperti nama, file, dan tanggal masuk.

2.2.1 Alasan Pemilihan Struktur Data

Struktur data *List* yang dimodifikasi sebagai *Stack* dipilih karena sangat representatif untuk memecahkan kasus "Tumpukan Dokumen Meja Kerja". Sistem ini mewajibkan dokumen yang paling terakhir kali diterima/ditambahkan (paling atas di tumpukan fisik) untuk diproses terlebih dahulu. Prinsip kerja *Last In, First Out* (LIFO) yang dimiliki Stack memfasilitasi kebutuhan ini tanpa harus melakukan perulangan kompleks. Operasi penambahan dan penghapusan di ujung tumpukan memberikan kompleksitas waktu amortisasi $O(1)$, yang sangat optimal untuk sistem dengan alur keluar-masuk arsip yang intens.

Struktur *Dictionary* dipilih karena karakteristik keterbacaan kodenya yang tinggi. Berbeda dengan *tuple* atau *array* dua dimensi yang mengharuskan pemanggilan atribut menggunakan indeks numerik yang sulit dilacak (seperti `data[0]` untuk nama, `data[1]` untuk tanggal), *Dictionary* memungkinkan pengembang memanggil atribut berdasarkan kuncinya (seperti `dokumen["tanggal"]`). Ini mempermudah pemeliharaan sistem, terutama pada fase modifikasi elemen maupun pertukaran parameter saat proses sortir..

2.3 Algoritma yang Digunakan

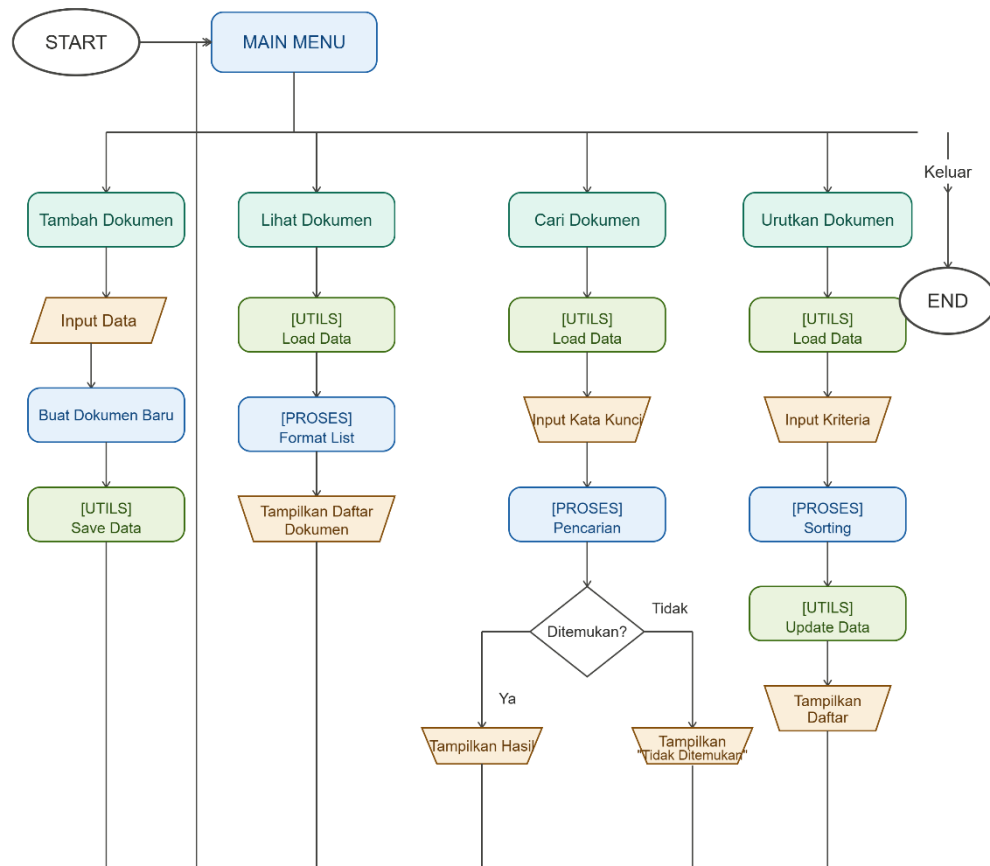
2.3.1 Searching

Algoritma yang digunakan adalah *Sequential Search* (pencarian berurutan). Algoritma *Sequential Search* diimplementasikan pada fungsi pencarian dokumen (*cari_menu*). Alasan utama digunakannya metode ini adalah arsitektur *Stack* yang tidak menuntut datanya untuk berada dalam kondisi terurut secara alfabetis. *Binary Search* akan gagal secara logis pada struktur *Stack* murni kecuali jika tumpukan tersebut dibongkar dan diurutkan terlebih dahulu, yang mana akan merusak sifat historis tumpukan itu sendiri. Dengan *Sequential Search*, program akan menyusuri tumpukan dari awal hingga akhir, mencocokkan substruktur string (*substring match*) yang tidak sensitif terhadap huruf kapital (*case-insensitive*). Berkat skala data dokumen meja kerja yang biasanya hanya mencakup puluhan atau ratusan file aktif, pencarian dengan kompleksitas $O(N)$ masih dieksekusi secara instan dan sangat mangkus.

2.3.2 Sorting

Algoritma pengurutan yang digunakan adalah *Merge Sort*. Algoritma *Merge Sort* diimplementasikan pada fitur layar Sorting yang memfasilitasi pelaporan terurut. Alasan utamanya adalah efisiensi Merge Sort yang sangat konsisten dengan kompleksitas $O(N \log N)$ pada semua jenis *worst*, *average*, maupun *best case*. Berbeda dengan *Bubble Sort* yang memiliki kompleksitas waktu kuadratik $O(N^2)$ dan rentan berjalan lambat bila tumpukan file membesar, *Merge Sort* dapat mengurutkan dengan pendekatan *Divide and Conquer* (membelah array list dan menggabungkannya). Selain itu, *Merge Sort* tergolong *stable sort*, yang memastikan bahwa apabila terdapat dua dokumen dengan kriteria (misalnya tanggal) yang sama, urutan relatif dari kedua dokumen tersebut tidak akan tertukar.

2.4 Diagram Alir Program (Flowchart)



Gambar 2.1 Diagram alir (flowchart) program

2.5 Struktur Program

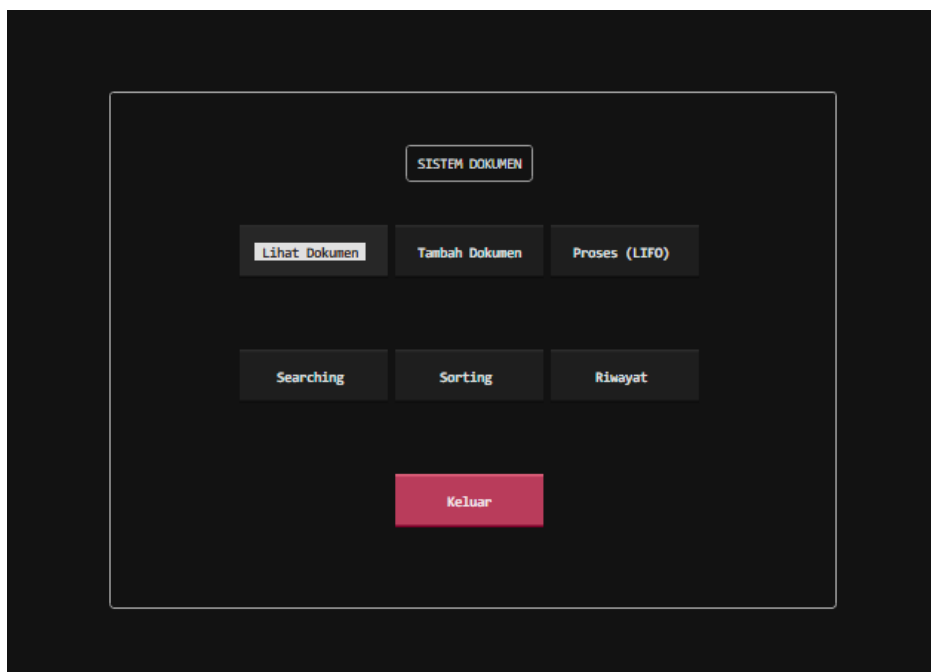
Aplikasi ini dibangun menggunakan arsitektur modular dengan struktur file yang saling berinteraksi secara spesifik.

file	Deskripsi
main.py	Sebagai entry point aplikasi, mendeklarasikan <i>App</i> , merender CSS, dan menjembatani routing berbagai layar menu utama.
tambah.py	Modul yang menangani antarmuka dan backend penambahan data (Create), termasuk memanggil API OS untuk jendela penjelajah fail fisik (file explorer).
lihat.py	Modul terpadu untuk mengeksekusi fitur Read (membaca tumpukan), Update (modifikasi parameter), dan Delete (penghapusan arsip dengan konversi indeks).

proses.py	Skrip inti dari eksekutor logika LIFO yang melakukan pop elemen Stack dan melempar sisa datanya ke modul I/O.
search.py	Modul spesifik yang berisi logika Sequential Search beserta tampilan TUI hasil temuan yang dikembalikan ke layar panggil.
sort.py	Modul spesifik algoritma Merge Sort untuk melakukan pengurutan alfabetis (A-Z) maupun kronologis (Tanggal lama ke baru).
utils.py	Berperan sebagai file handler yang menjembatani I/O ke penyimpanan lokal; memuat fungsi komputasi <code>baca_data()</code> , <code>simpan_data()</code> , dan <code>tambah_riwayat()</code> .
data.txt	File basis data yang menyimpan antrean aktif dokumen.
riwayat.txt	File log untuk mencatat seluruh arsip yang telah selesai diproses oleh fungsi LIFO

BAB III IMPLEMENTASI PROGRAM

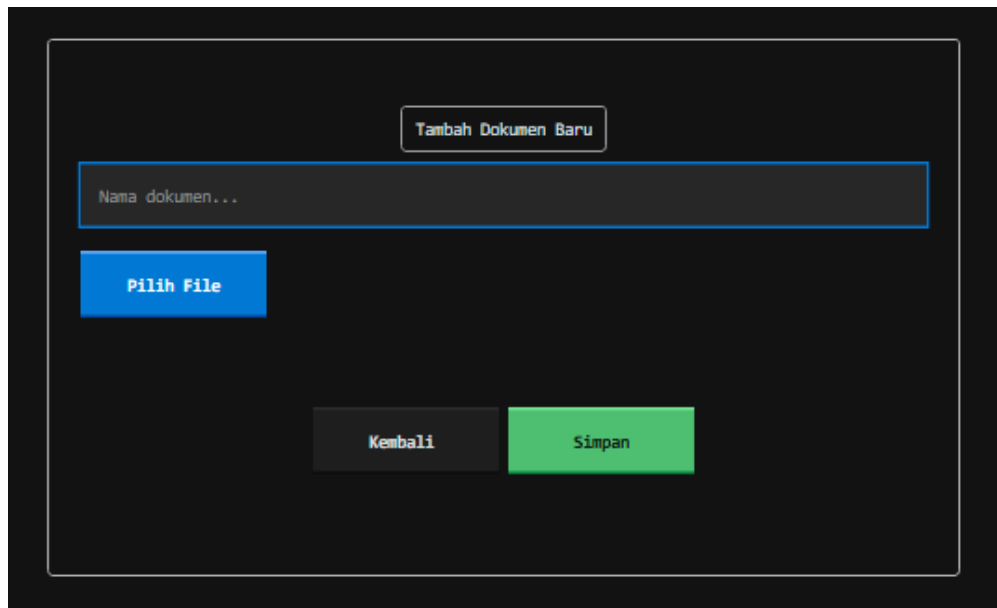
3.1 Tampilan Menu Utama



Gambar 3.1 Tampilan menu utama aplikasi

Aplikasi menggunakan *textual.app* untuk merancang menu antar muka yang meniru jendela GUI klasik namun dikonstruksi dari karakter terminal. Elemen diletakkan dalam *Horizontal* dan *Vertical containers*, yang dilengkapi dengan *Button* untuk tiap modul fungsi.

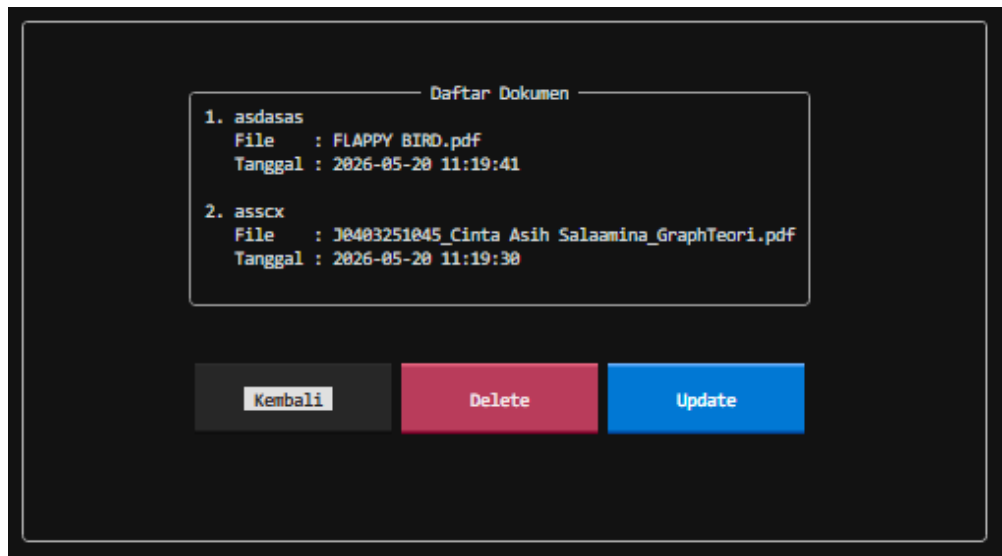
3.2 Fitur Create



Gambar 3.2 Fitur Create — Tambah dokumen Baru

Fitur penambahan dokumen (*Create*) dirancang dengan dua komponen masukan: teks nama dokumen dan path direktori fisik. Menggunakan jendela dialog sistem natif lewat perpustakaan standar *tkinter* (*filedialog*), pengguna dapat memilih file berekstensi *.pdf* atau *.docx*. Setelah verifikasi berhasil, skrip memanggil *shutil.copy()* untuk menggandakan berkas tersebut ke direktori *file_dokumen/*. Entitas baru lantas dicatat dalam struktur *Dictionary*, disertai penyematan *timestamp* tanggal komputasi via *datetime.now()*, lalu didorong ke dalam List (*Stack*) menggunakan *data.append()*.

3.3 Fitur Read



Gambar 3.3 Fitur Read — Tampilan daftar dokumen

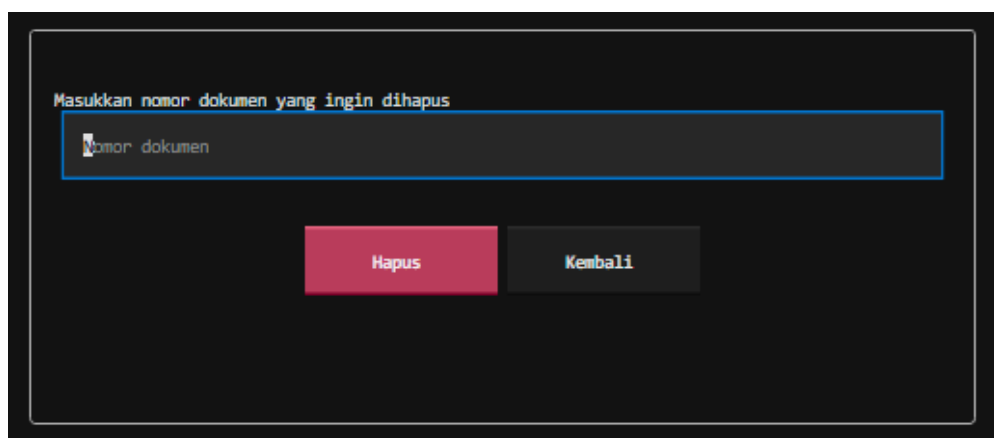
Fitur pemanggilan (Read) diakses melalui antarmuka *LihatDokumenScreen*. Saat panel dirender, skrip membaca struktur di memori kemudian menampilkan daftar tabel dari yang terbaru ke terlama secara visual dengan perulangan bawaan *reversed(data)*. Tujuannya adalah mensimulasikan tumpukan dokumen fisik di mana barang terakhir masuk akan langsung terlihat secara jelas pada antrean di nomor 1 teratas.

3.4 Fitur Update

Gambar 3.4 Fitur Update — Ubah nama / file dokumen

Pembaruan data difasilitasi dalam modul khusus *UpdateScreen*. Karena secara visual urutan telah dibalik oleh fitur *Read*, ketika pengguna ingin memperbarui dokumen bernomor 1 (objek terakhir di memori), logika internal harus melakukan perhitungan indeks mundur. Skrip mengeksekusi $real_index = len(data) - index$. Apabila $real_index$ berhasil divalidasi, parameter dalam kamus seperti $data[real_index][nama]$ akan ditimpa dengan nilai (*value*) baru dan *simpan_data()* dipanggil.

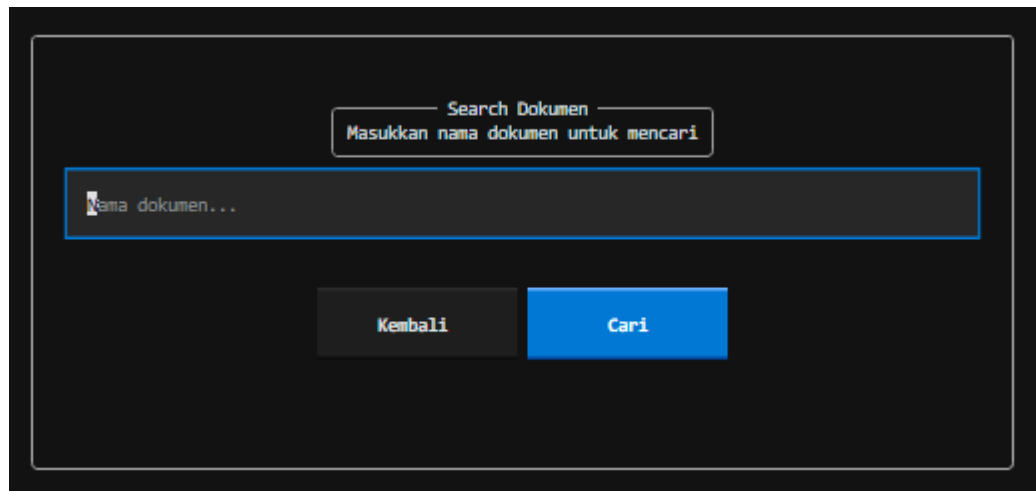
3.5 Fitur Delete



Gambar 3.5 Fitur Delete — Hapus dokumen

Serupa dengan fitur *Update*, penghapusan elemen secara bebas diimplementasikan menggunakan konversi indeks (*reversed indexing*). Sistem membaca nomor urut yang diketik oleh pengguna, mengubah string input menjadi integer, memvalidasi jangkauan array (menghindari *IndexError*), dan menggunakan fungsi $data.pop(len(data) - index)$ untuk memotong ikatan elemen tersebut dari memori utama Stack, tanpa memindahkannya ke riwayat (dianggap ditarik paksa).

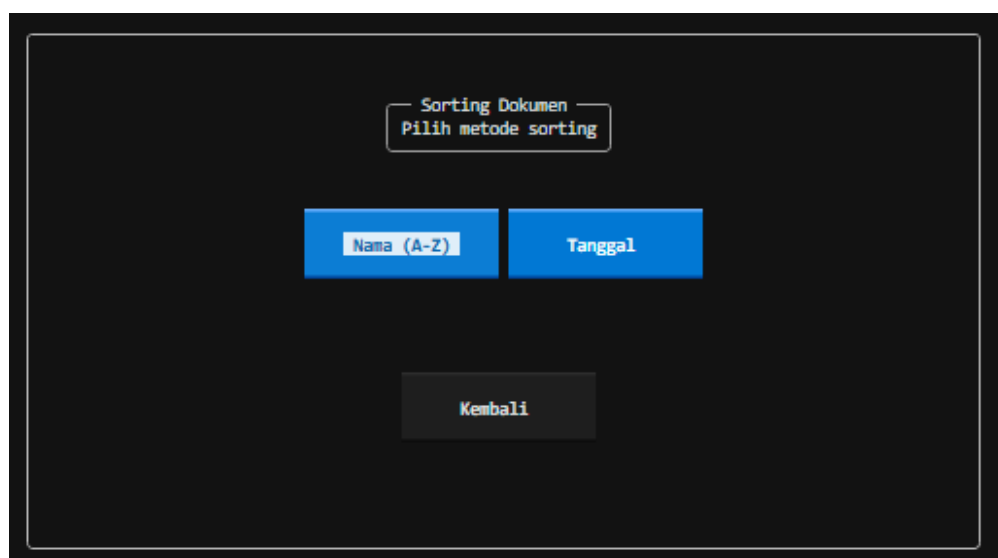
3.6 Fitur Searching



Gambar 3.6 Fitur Searching — Pencarian menu berdasarkan kata kunci

Menggunakan kelas *HasilSearchScreen*, pencarian data diimplementasikan dengan algoritma *Sequential Search*. Algoritma ini berjalan pada perulangan *for d in data* dengan komparator bersyarat yang menggabungkan *lower()* dan operator keanggotaan *in* (sebagai contoh: *if keyword in d[nama].lower():*). Pendekatan *partial match* ini mengizinkan pengguna untuk sekadar mengetikkan sebagian kata benda dan segera memperoleh entitas lengkap dokumen yang terhimpun ke dalam senarai `'hasil = []'` baru.

3.7 Fitur Sorting

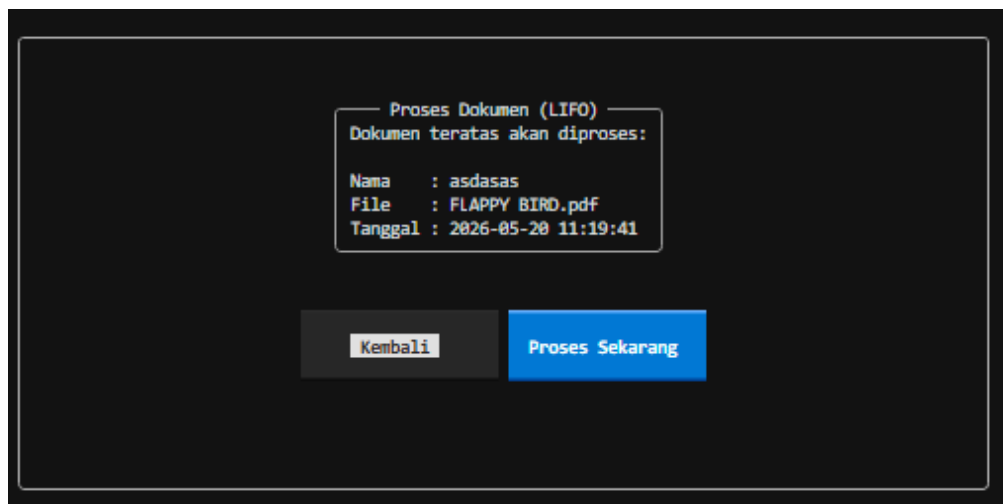


Gambar 3.7 Fitur Sorting — Pengurutan dokumen

Fitur ini menyajikan opsi pengurutan melalui antarmuka layar tersendiri. Ketika tombol ditekan, list data yang di-load dari *baca_data()* dikirim ke fungsi *merge_sort()*.

Algoritma rekursif *Merge Sort* akan secara kontinu membelah *array* hingga satu elemen, membandingkan kunci atribut (seperti nama atau tanggal), dan menggabungkan (*merge*) blok tersebut. Hasil penggabungan kemudian dikonstruksi ulang menjadi daftar *string* berurut yang diprint (ditampilkan) pada panel *Rich*.

3.8 Fitur Proses Tumpukan



Gambar 3.8 Fitur Proses Tumpukan

Bagian pemrosesan tumpukan (LIFO) dioperasikan penuh melalui perintah *Proses Sekarang*. Program memuat senarai *data*, kemudian tanpa konfirmasi pilihan nomor mengaktifkan metode *dokumen = data.pop()*. Metode ini menjamin bahwa objek array yang selalu terekstraksi dan dilepas dari memori list adalah elemen absolut paling ujung (paling terbaru). Objek dokumen yang tereliminasi tersebut lalu dikirim pada prosedur *tambah_riwayat(dokumen)* yang mana objek kamus tersebut disematkan pada baris paling akhir di *riwayat.txt* dalam operasi mode *append* (a).

3.9 Penyimpanan Data (File Handling)

Infrastruktur penyimpanan dan persistensi data sepenuhnya ditangani oleh `utils.py`. Model basis data yang digunakan adalah file TXT datar (Flat Text File). Format ini mempermudah migrasi perangkat dan memastikan aplikasi tetap ringan dan tidak bergantung pada RDBMS (Relational Database Management System).

- Format file yang digunakan: File teks murni berformat `.txt` (UTF-8 encoding) yang direpresentasikan dalam `data.txt` dan `riwayat.txt`.
- Cara menyimpan file: Dilakukan melalui konteks `with open (DATA_FILE, "w") as file`. Seluruh list Dictionary diiterasi menggunakan `for loop`, setiap parameter kunci direkatkan menggunakan teknik interpolasi f-string yang disekat oleh karakter pipa tegak (`|`). Mode `"w"` digunakan untuk menimpa ulang (`overwrite`) state lama setiap terjadi interaksi.
- Cara membaca file: Membaca sebaris demi sebaris via `readlines()`, lalu diurai menjadi senarai variabel komponen menggunakan metode `line.strip().split("|")` yang kemudian dipetakan ulang kembali menjadi objek Kamus.

Contoh isi file `data.txt`:

```
asscx|J0403251045_Cinta Asih Salaamina_GraphTeori.pdf|2026-05-20 11:19:30
asdasas|FLAPPY BIRD.pdf|2026-05-20 11:19:41
5
```

3.10 Validasi Input dan Error Handling

Untuk menjaga kestabilan program, berbagai mekanisme validasi input dan penanganan kesalahan (*error handling*) diterapkan pada seluruh bagian aplikasi, sebagaimana dirangkum pada Tabel 3.1 berikut.

Kesalahan Input	Penanganan
Huruf dimasukkan pada input angka ID saat proses Update/Delete	Skrip memvalidasi dengan ekspresi <code>not nomor.isdigit()</code> . Program menangkal Type Error dan secara elegan meminta pengguna menginput angka.
Data tidak ditemukan / indeks di luar jangkauan tumpukan	Fungsi validasi <code>if index < 1 or index > len(data)</code> menangkap Index Out of Bounds Error dan pesan "Nomor tidak valid" ditampilkan.

File ekstensi sembarang yang tidak didukung masuk dari antarmuka OS	Kondisional not (<code>file_path.endswith(".pdf")</code> or <code>file_path.endswith(".docx")</code>) akan mengembalikan string <code>error*</code> ke Widget Static TUI dan menghentikan pengunggahan file tersebut.
File basis data atau direktori <code>file_dokumen</code> belum tersedia saat instalasi baru	Modul utilitas dan modul tambah dibekali skrip mitigasi otomatis via modul OS: <code>if not os.path.exists("file_dokumen"):</code> <code>os.makedirs("file_dokumen")</code> . Hal ini menciptakan folder kosong dan menghindari fatal <code>Directory Not Found Exception</code> .

BAB IV PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian kotak hitam (*Black Box Testing*) dilakukan dengan mengevaluasi *behavior* fungsional interaktif TUI ketika menerima masukan skenario positif dan negatif.

No	Skenario	Tindakan Sistem dan Kondisi	Hasil
1	Tambah data	Input teks terisi, memilih file DOCX dari penyimpanan, data ditambahkan.	Berhasil. File tercopy dan parameter tercatat di <code>data.txt</code> .
2	Edit Data	Melakukan pembaruan Nama pada dokumen Nomor 2. Sistem melakukan reverse-index	Berhasil. Objek index ke-2 berubah tanpa mengintervensi objek sekitarnya.
3	Hapus Data	Menghapus dokumen melalui indeks yang dikonversi dari list visual terbalik.	Berhasil. Dokumen hilang spesifik pada antrean yang diinputkan.
4	Sorting Data	Mengeksekusi Merge Sort berdasarkan Tanggal untuk mengurutkan file.	Berhasil. Susunan tanggal paling lama terproyeksi sesuai prioritas atas tanpa crash.

5	Searching Data	Mencari substring kecil (misal "lap") pada senarai dokumen panjang secara case-insensitive.	Berhasil. Semua dokumen mengandung kata "lap" teragregasi secara parsial.
6	Proses LIFO (Empty)	Menekan tombol "Proses Sekarang" ketika database data.txt kosong bersih.	Berhasil. Pop Error dicegah oleh penanganan kondisi if not data, muncul warning.

4.2 Kelebihan Sistem

1. Data tersimpan permanen dan andal. Berkat implementasi Context Manager berkas otomatis (I/O) setiap selesai operasi, resiko manipulasi RAM yang menguap sepenuhnya hilang.
2. Program tidak mudah crash. Kombinasi metode validasi TUI Textual dengan penanganan galat logika index boundary limits memproteksi tumpukan Stack dari interupsi manual atau ValueError.
3. Menu sangat mudah digunakan. Pendekatan penggunaan antarmuka Text-based GUI menyembunyikan kelemahan CLI mentah, memberikan tombol klik terpadu (mouse support) serta visual garis tepi (border) yang elegan.
4. Performa Instan LIFO $O(1)$. Kecepatan pemrosesan tugas Stack Python sangat teroptimasi sehingga tidak menyedot performa CPU berlebih.

4.3 Keterbatasan Sistem

1. Masih berbasis CLI hibrida. Walaupun ditenagai pustaka TUI Textual, program tetap terikat pada fungsionalitas rendering konsol. Program sangat rentan mengalami pergeseran tata letak layout jika pengguna mengecilkan resolusi jendela terminal dengan rasio yang ekstrem.
2. Belum mendukung database relasional. Flat file .txt dengan delineator sangat berisiko korup jika suatu waktu nama dokumen inputan pengguna kebetulan memiliki lambang karakter delineator (|) itu sendiri.
3. Belum mendukung multiuser dan sinkronisasi Cloud. Implementasi ini merupakan model standalone executable di penyimpanan lokal tunggal, sehingga jika meja kerja dioperasikan oleh lebih dari satu orang administrator secara simultan di PC berbeda, data tidak akan saling terkoneksi.

4. Tidak memiliki pengelola sampah fisik. Berkas fisik di `file\_dokumen/` akan menumpuk dari waktu ke waktu karena fitur Delete dan LIFO hanya menghilangkan rincian teks objek dari kamus database `txt`, sementara utilitas untuk membersihkan berkas fisiknya secara absolut via `os.remove()` belum diprogram.

BAB V KENDALA DAN SOLUSI

Kendala	Solusi
Data hilang saat program ditutup secara mendadak (<i>Force Close</i>).	Menggunakan file TXT sebagai pangkalan data sekunder. Sistem menyimpan otomatis (<i>auto-save</i>) dengan melakukan pembacaan berulang setiap perpindahan <i>Screen</i>
Program error (<i>IndexError</i> / <i>TypeError</i>) saat input urutan data salah di modul modifikasi	Menambahkan validasi kondisional <code>.isdigit()</code> dan pembatasan batas nilai minimum-maksimum menggunakan operator logis perbandingan <code>len()</code> data array
Sorting berjalan lambat atau rentan tertukar referensinya pada pencocokan array of string.	Mengganti algoritma dasar dengan algoritma yang lebih efisien yakni <i>Merge Sort</i> ($O(N \log N)$) yang dijamin menjaga <i>stable sequence</i> objek aslinya
Layar TUI tertutup panel kosong dari pustaka modul OS pemilih file (Tkinter GUI).	Menyisipkan komando <code>root.withdraw()</code> tepat sebelum meluncurkan sub-menu <code>filedialog</code> untuk menghancurkan render latar jendela OS

Pengalaman komputasi yang paling menantang bagi tim terletak pada transisi pemahaman logis "Nomor Antrean Layar" dengan "Indeks Real Array". Karena prinsip dasar operasional sistem ini adalah LIFO (Last In, First Out), dokumen terakhir yang dimasukkan (indeks terakhir Python) harus disajikan pada urutan nomor teratas (Nomor 1) di antarmuka agar terasa logis secara fungsional. Proses membalik array untuk tampilan visual ternyata menciptakan bug krusial; saat

administrator mengetik "Hapus dokumen No. 1", fungsi list standar di memori malah akan membidik dokumen usang (indeks awal array sesungguhnya).

Tantangan tersebut menuntut kolaborasi desain logika matematis sederhana yang berorientasi abstraksi. Tim akhirnya memecahkannya secara solid dengan menciptakan metode kalkulasi konversi (*reversed-indexing manipulation*) yakni dengan merumuskan: $\text{Indeks_Nyata} = \text{Total Panjang Array} - \text{Angka Inputan Pengguna}$. Pendekatan kalkulatif ini menyelesaikan masalah intervensi memori yang membingungkan dengan sangat elegan tanpa mengurangi responsivitas aplikasi Textual.

BAB VI PEMBAGIAN TUGAS ANGGOTA

Nama	Tugas
Cinta Asih Salaamina	Project Manager. Mengoordinasikan jalannya proyek, mengimplementasikan <i>import library</i> utama yang dibutuhkan aplikasi, serta merancang modul antarmuka utama, utilitas, dan fitur baca/tampil data (<i>main.py</i> , <i>utils.py</i> , <i>lihat.py</i>)
Najwa Hanifa Sholiha Asy Syifa SM	Riset Struktur Data. Menganalisis dan menentukan struktur data yang paling efisien untuk sistem, serta mengembangkan fungsionalitas penambahan dokumen dan algoritma pengurutan (<i>tambah.py</i> , <i>sort.py</i>)
Selena Rohmani Putri	Perancang Flowchart. Mendesain diagram alir (<i>flowchart</i>) untuk memetakan alur logika program, serta membangun fungsionalitas pencarian spesifik dan pemrosesan utama tumpukan dokumen LIFO (<i>search.py</i> , <i>proses.py</i>).

Secara keseluruhan, setiap anggota kelompok telah berkontribusi dengan aktif baik pada aspek manajerial, perancangan logika, maupun penulisan kode. Cinta bertindak sebagai penggerak utama proyek yang memastikan seluruh *library* pendukung terintegrasi dengan baik dan *routing* antarmuka berjalan lancar. Najwa berfokus pada fondasi penyimpanan di dalam memori melalui pemilihan struktur data yang tepat, sekaligus memastikan fitur penginputan dan penyusunan (*sorting*) arsip berfungsi tanpa kendala. Sementara itu, Selena menerjemahkan kerumitan operasional ke dalam *flowchart* yang komprehensif, serta membidani fungsi kritikal

untuk menarik (*pop*) dokumen dari antrean dan mencari dokumen spesifik menggunakan algoritma yang efisien.

BAB VII KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil pengerjaan dan pembahasan proyek ini, maka dapat disimpulkan bahwa:

1. Keberhasilan pembuatan aplikasi Manajemen Dokumen berbasis terminal menggunakan kerangka Textual dan struktur data Python terbukti mampu meniru siklus sistem tata kelola LIFO (Last-In, First-Out) meja kerja administratif fisik ke dalam antarmuka interaktif yang kokoh.
2. Implementasi model Stack (Tumpukan) List dipadukan Dictionary untuk penyimpanan metadata berhasil mempercepat eksekusi pemrosesan dokumen prioritas, berjalan stabil dengan operasi waktu $O(1)$, dan mampu menangkal crash aplikasi secara efektif.
3. Sistem berhasil dimampukan untuk melakukan pertahanan persistensi data operasional (stateless save state) melalui prosedur penyimpanan otomatis I/O Stream File Handling teks datar (flat file) sehingga riwayat dan status manipulasi (CRUD) tidak terdampak oleh kehilangan daya memori lokal.

7.2 Saran Pengembangan

1. Pengembangan Automasi Garbage Cleaner Fisik: Pada versi selanjutnya, program diharapkan dibekali baris kode instruksi Operating System (melalui modul `os.remove()`) yang dapat memburu dan menghapus residu file .pdf dan .docx lokal di direktori manakala status data mereka telah dipotong dari array log.
2. Transisi Tipe Penyimpanan Terstruktur: Melakukan modernisasi pada pengelola pangkalan data dari format flat txt separator beralih menggunakan basis data SQLite ringan atau standardisasi serialisasi format JavaScript Object Notation (JSON) agar terhindar dari corrupted string match.
3. Pengembangan Fitur Multi-Queue (Antrean Jamak): Menambah fungsionalitas manajemen penanda Priority Flag (Misal: Dokumen

Mendesak, Dokumen Rutin) dengan menginjeksi algoritma Priority Queue yang berjalan secara paralel di dalam wadah memori Stack LIFO utama.

DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms, 4th ed. MIT Press, Cambridge, Massachusetts.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. John Wiley & Sons, Inc.
- Mcgugan, W. (2024). Textual Documentation: A Rapid Application Development framework for Python. PyPI Repositories Documentation.
- Miller, B., & Ranum, D. (2011). Problem Solving with Algorithms and Data Structures Using Python. Franklin, Beedle & Associates.
- Python Software Foundation. (2024). The Python Language Reference (List Objects & File Handling). Diakses secara daring pada dokumentasi perpustakaan resmi Python 3.

LAMPIRAN

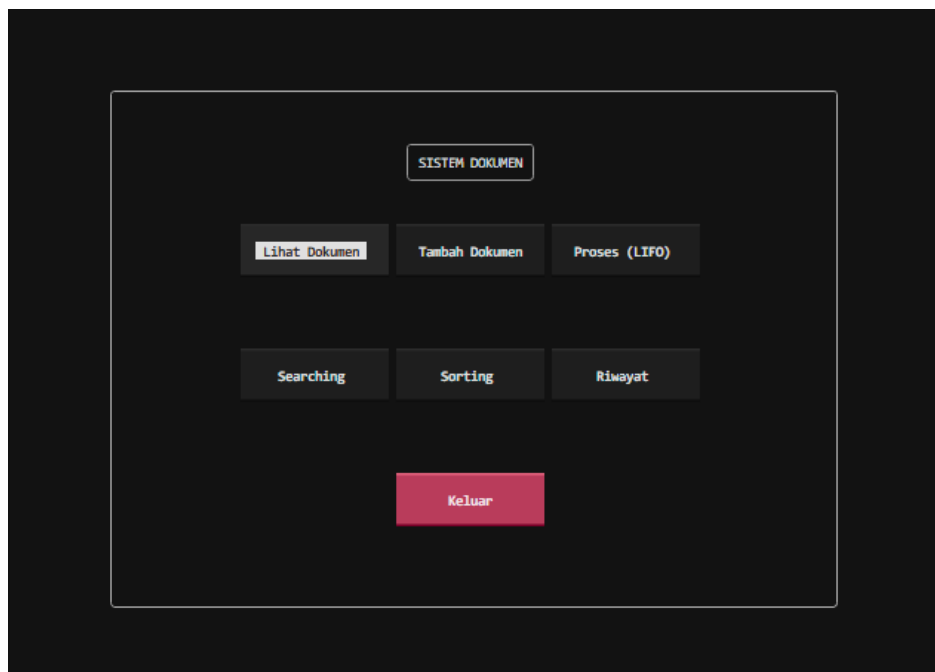
Lampiran A. Link GitHub

Repositori kode sumber lengkap proyek ini dapat diakses melalui tautan berikut:

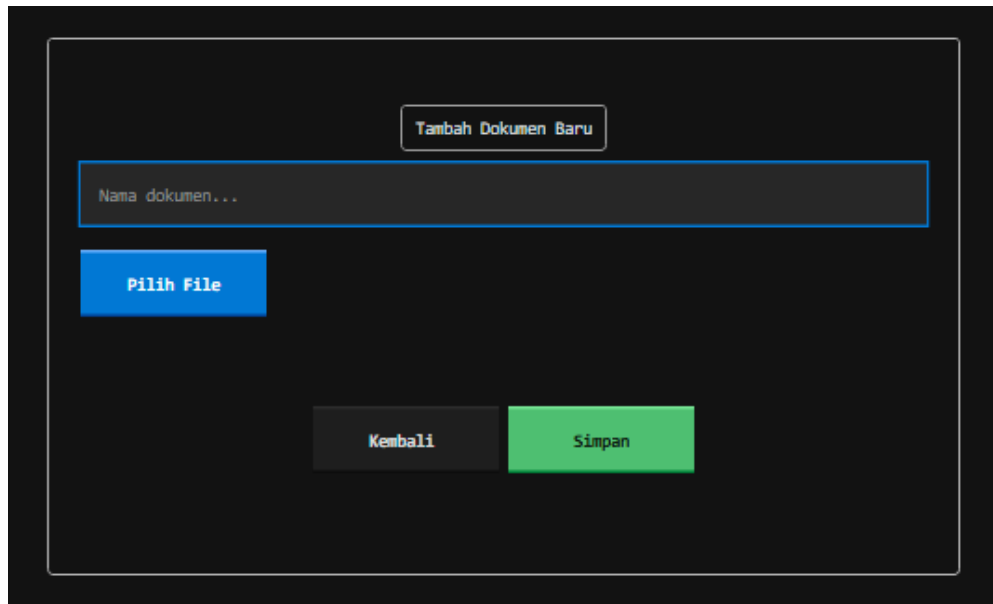
https://github.com/casalaamina/ASD_B_8

Lampiran B. Screenshot Program

1. Tampilan Menu Utama



1. Fitur Create



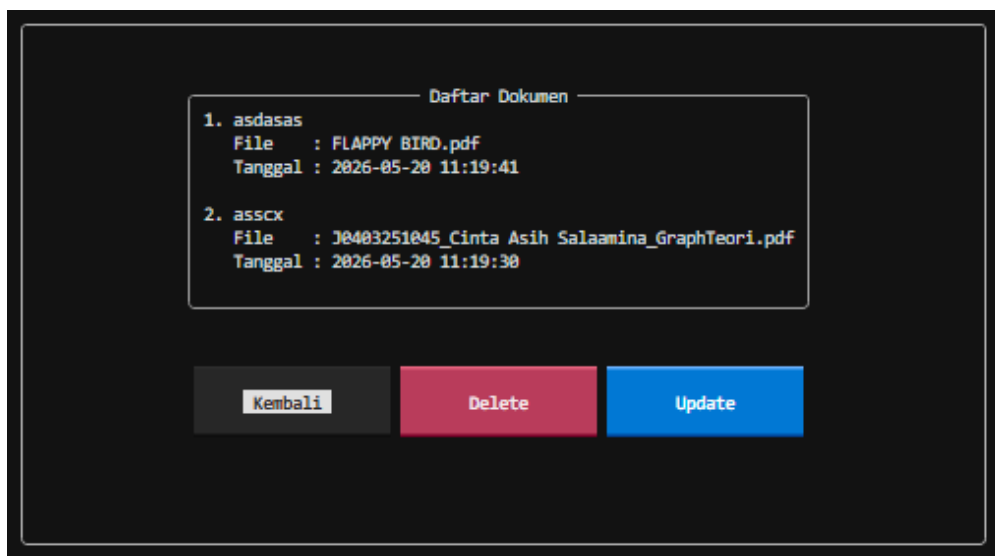
Tambah Dokumen Baru

Nama dokumen...

Pilih File

Kembali Simpan

2. Fitur Read

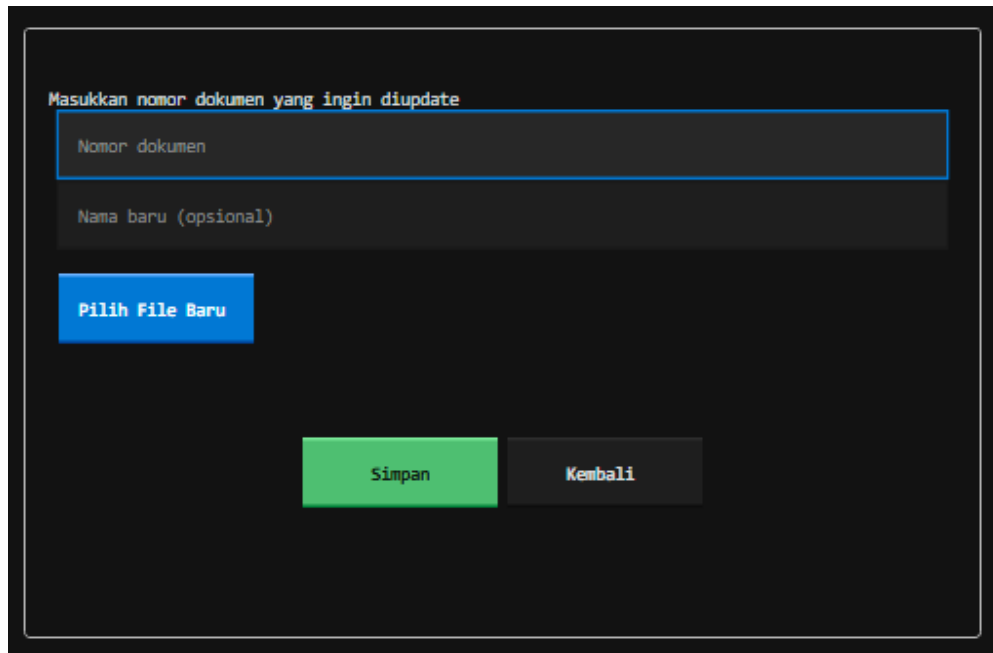


Daftar Dokumen

1. asdasas
File : FLAPPY BIRD.pdf
Tanggal : 2026-05-20 11:19:41
2. asscx
File : J0403251045_Cinta Asih Salaamina_GraphTeori.pdf
Tanggal : 2026-05-20 11:19:30

Kembali Delete Update

3. Fitur Update



Masukkan nomor dokumen yang ingin diupdate

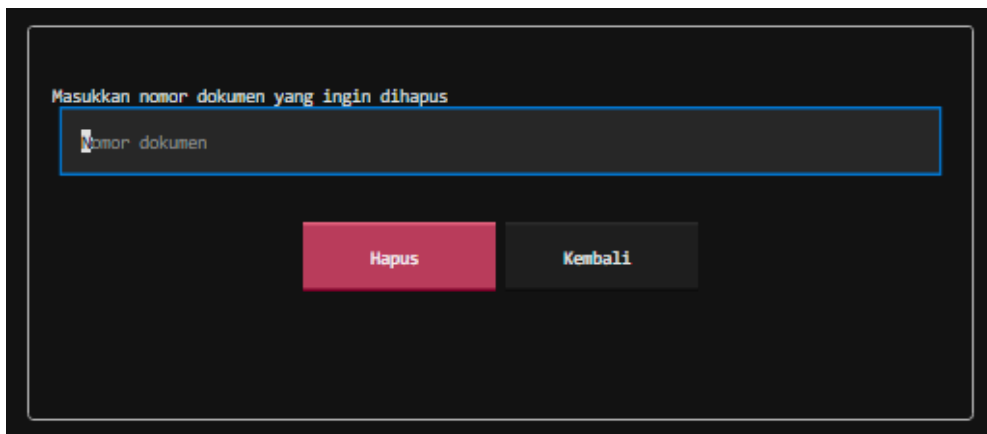
Nomor dokumen

Nama baru (opsional)

Pilih File Baru

Simpan Kembali

4. Fitur Delete

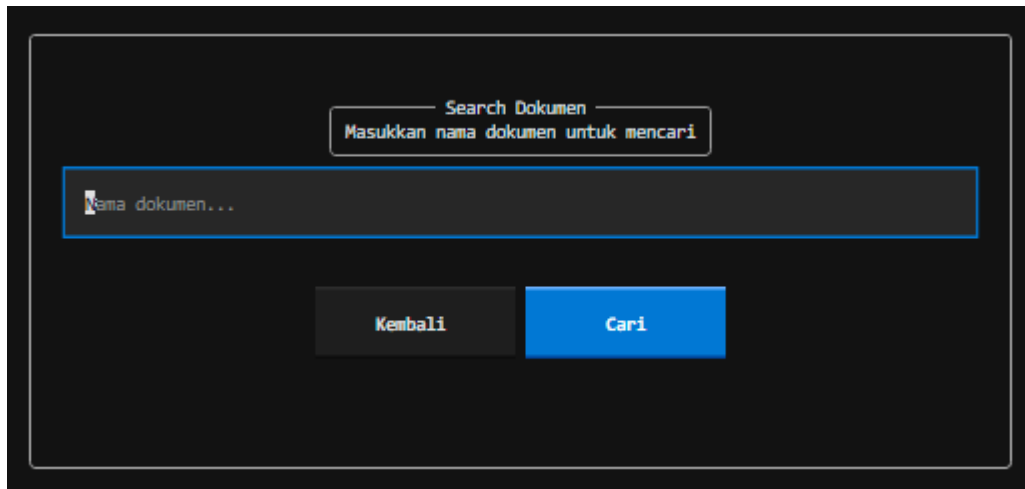


Masukkan nomor dokumen yang ingin dihapus

Nomor dokumen

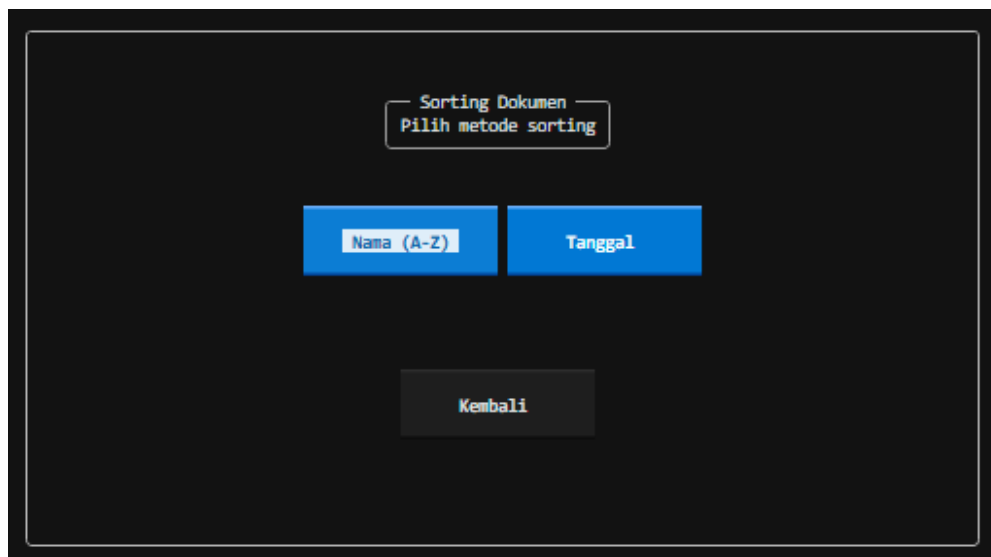
Hapus Kembali

5. Fitur Searching



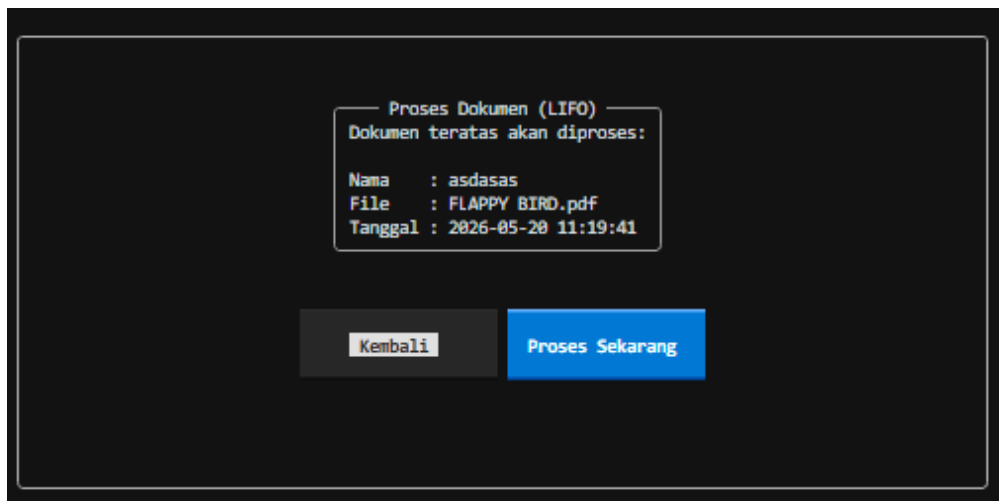
The screenshot shows a dark-themed user interface for searching documents. At the top, there is a title "Search Dokumen" and a subtitle "Masukkan nama dokumen untuk mencari". Below this is a large, dark text input field with the placeholder text "Nama dokumen...". At the bottom of the interface, there are two buttons: a dark button labeled "Kembali" (Back) and a blue button labeled "Cari" (Search).

6. Fitur Sorting



The screenshot shows a dark-themed user interface for sorting documents. At the top, there is a title "Sorting Dokumen" and a subtitle "Pilih metode sorting". Below this are two blue buttons: "Nama (A-Z)" and "Tanggal" (Date). At the bottom of the interface, there is a dark button labeled "Kembali" (Back).

7. Fitur Proses Tumpukan



Lampiran C. Struktur Folder Proyek

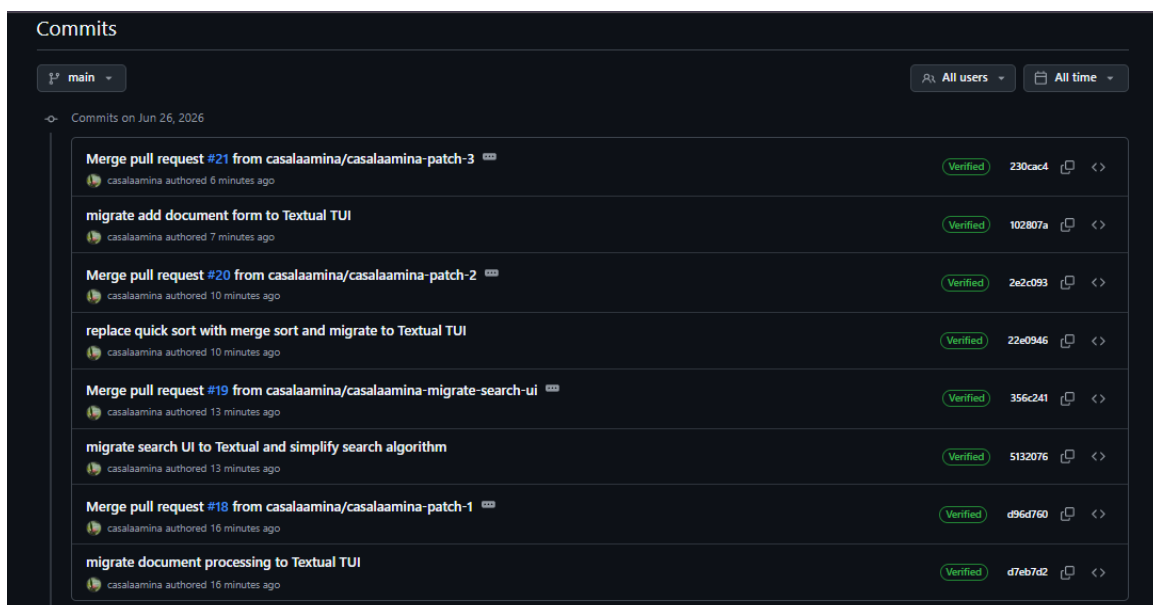
Struktur direktori

```

sistem_manajemen_dokumen/
├── file_dokumen/
│   ├── J0403251045_Cinta Asih_GraphTeori.pdf # Direktori khusus menyimpanna file PDF/DOCX fisik
│   └── Buku Kerja UTS_Logika.pdf # Direktori khusus menyimpanna GraphTeori.pdf
├── data.txt # Buku Kerja UTS_Logika.pdf format 12=l.pdf
├── riwayat.txt # Basis data LIFO (Teks datar terstruktur)
├── main.py # Log rekam jejak LIFO (Dokumen telah diproses)
├── utils.py # Titik masuk program (Entry Point)
├── tambah.py # Modul fungsi bantuan I/O (File Handler)
├── lihat.py # Modul fungsi Create / Push Stack
├── proses.py # Modul antarmuka Read, Update, Delete
├── search.py # Modul utilitas Pop LIFO eksekutor
├── sort.py # Modul pencarian dengan Sequential Search
└── README.md # Modul algoritma pengurutan dengan Merge Sort
# Dokumentasi kompilasi proyek di GitHub

```

Lampiran D. Commit History GitHub



Commits on May 20, 2026	
Merge pull request #17 from casalaamina/casalaamina-utils	Verified d1738fc
casalaamina authored on May 20	
Refactor data handling and add formatting function	Verified c7be132
casalaamina authored on May 20	
Merge pull request #16 from casalaamina/casalaamina-lihat-dokumen	Verified 51e3297
casalaamina authored on May 20	
Refactor document viewing and management with GUI	Verified bdd3439
casalaamina authored on May 20	
Merge pull request #15 from casalaamina/casalaamina-ui-main	Verified 6eb6506
casalaamina authored on May 20	
Refactor main.py to use Textual for UI	Verified 38f53af
casalaamina authored on May 20	
Commits on May 13, 2026	
Merge pull request #14 from casalaamina/casalaamina-update-sort	Verified b4a415f
casalaamina authored on May 13	
Merge pull request #13 from casalaamina/casalaamina-update-search	Verified bf5b3b3
casalaamina authored on May 13	
update-metode-sort	Verified 0ab4c9e
casalaamina authored on May 13	
update-metode-search	Verified 4605198
casalaamina authored on May 13	
Commits on Apr 15, 2026	

Commits on Apr 15, 2026	
Merge pull request #12 from casalaamina/csnoirss-FiturTambah	Verified e2e3482
csnoirss authored on Apr 15	
Fitur Tambah	Verified de7e558
csnoirss authored on Apr 15	
Merge pull request #11 from casalaamina/csnoirss-FiturSort	Verified 367d1ef
csnoirss authored on Apr 15	
Fitur Sort	Verified 45f22d1
csnoirss authored on Apr 15	
Merge pull request #10 from casalaamina/senaelenaa-Fitur-Proses	Verified c04be4b
senaelenaa authored on Apr 15	
Add files via upload	Verified cb529e9
senaelenaa authored on Apr 15	
Merge pull request #9 from casalaamina/senaelenaa-FiturSearch	Verified e750fac
senaelenaa authored on Apr 15	
Fitur Search	Verified d580f86
senaelenaa authored on Apr 15	
Merge pull request #1 from casalaamina/casalaamina-menuAwal-read-utils	Verified 7d80233
casalaamina authored on Apr 15	
Add files via upload	Verified 4aactbf6
casalaamina authored on Apr 15	
Commits on Feb 11, 2026	
Initial commit	Verified f95882e
casalaamina authored on Feb 11	

Lampiran E. Source Code Utama

main.py

```
from textual.app import App, ComposeResult
from textual.widgets import Header, Footer, Button, Static
from textual.containers import Vertical, Horizontal
from textual.screen import Screen
from rich.panel import Panel
from rich.align import Align

from tambah import tambah_dokumen
from lihat import lihat_dokumen
from proses import proses_dokumen
from search import search_dokumen
from sort import sort_dokumen
from utils import baca_riwayat

class MenuScreen(Screen):
    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(Panel("SISTEM DOKUMEN", expand=False))),
            Horizontal(
                Button("Lihat Dokumen", id="lihat"),
                Button("Tambah Dokumen", id="tambah"),
                Button("Proses (LIFO)", id="proses"),
            ),
            Horizontal(
                Button("Searching", id="search"),
                Button("Sorting", id="sort"),
                Button("Riwayat", id="riwayat"),
            ),
            Horizontal(
                Button("Keluar", id="keluar", variant="error"),
            ),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "lihat":
            lihat_dokumen(self.app)

        elif event.button.id == "tambah":
            tambah_dokumen(self.app)

        elif event.button.id == "proses":
            proses_dokumen(self.app)

        elif event.button.id == "search":
            search_dokumen(self.app)

        elif event.button.id == "sort":
            sort_dokumen(self.app)

        elif event.button.id == "riwayat":
            self.app.push_screen(RiwayatScreen())

        elif event.button.id == "keluar":
            self.app.exit()

class RiwayatScreen(Screen):
    def compose(self) -> ComposeResult:
        raw = baca_riwayat()

        yield Header()

        if not raw:
            yield Vertical()
```

```

        Static(Panel("Belum ada riwayat.", expand=False)),
        Button("Kembali", id="back"),
    )
    yield Footer()
    return

items = []

for i, item in enumerate(raw, 1):
    if isinstance(item, dict):
        nama = item.get("nama", "-")
        tanggal = item.get("tanggal", "-")
        file = item.get("file", "-")

        content = (
            f>Nama : {nama}\n"
            f>Tanggal : {tanggal}\n"
            f>File : {file}"
        )
    else:
        content = str(item)

    items.append(
        Static(Panel(content, title=f"Dokumen {i}", expand=False))
    )

    yield Vertical(
        *items,
        Horizontal(
            Button("Kembali", id="back"),
        )
    )

    yield Footer()

def on_button_pressed(self, event: Button.Pressed) -> None:
    if event.button.id == "back":
        self.app.pop_screen()

class DokumenApp(App):
    CSS = """
    Screen {
        align: center middle;
    }

    Vertical {
        width: 60%;
        height: auto;
        align: center middle;
        padding: 2;
        border: round white;
    }

    Horizontal {
        height: auto;
        align: center middle;
        padding: 1;
    }

    Button {
        margin: 1;
        width: 20;
    }
    """

    def on_mount(self) -> None:
        self.push_screen(MenuScreen())

```

```
if __name__ == "__main__":
    DokumenApp().run()
```

utils.py

```
import os

DATA_FILE = "data.txt"
RIWAYAT_FILE = "riwayat.txt"

def baca_data():
    if not os.path.exists(DATA_FILE):
        return []

    with open(DATA_FILE, "r") as file:
        lines = file.readlines()

    data = []
    for line in lines:
        parts = line.strip().split("|")
        if len(parts) == 3:
            nama, file_doc, tanggal = parts
            data.append({
                "nama": nama.strip(),
                "file": file_doc.strip(),
                "tanggal": tanggal.strip()
            })
    return data

def simpan_data(data):
    with open(DATA_FILE, "w") as file:
        for d in data:
            file.write(f'{d["nama"]} | {d["file"]} | {d["tanggal"]} \n')

def tambah_riwayat(dokumen):
    with open(RIWAYAT_FILE, "a") as file:
        file.write(f'{dokumen["nama"]} | {dokumen["file"]} | {dokumen["tanggal"]} \n')

def baca_riwayat():
```



```

if not os.path.exists(RIWAYAT_FILE):

    return []

with open(RIWAYAT_FILE, "r") as file:

    lines = file.readlines()

data = []

for line in lines:

    parts = line.strip().split("|")

    if len(parts) == 3:

        nama, file_doc, tanggal = parts

        data.append({

            "nama": nama.strip(),

            "file": file_doc.strip(),

            "tanggal": tanggal.strip()

        })

data.reverse()

return data

def format_riwayat(riwayat):

    if not riwayat:

        return []

    max_nama = max(len(r["nama"]) for r in
riwayat)

    max_file = max(len(r["file"]) for r in
riwayat)

    max_tanggal = max(len(r["tanggal"]) for r in
riwayat)

    hasil = []

    header = f'{"NAMA".ljust(max_nama)} |
{"FILE".ljust(max_file)} |
{"TANGGAL".ljust(max_tanggal)}'

    garis = "-" * len(header)

    hasil.append(header)

    hasil.append(garis)

    for r in riwayat:

```

```

        baris = f'{r['nama'].ljust(max_nama)} |'
        {r['file'].ljust(max_file)} |
        {r['tanggal'].ljust(max_tanggal)} "

        hasil.append(baris)

    return hasil

```

lihat.py

```

from textual.app import ComposeResult
from textual.screen import Screen
from textual.widgets import Header, Footer, Static, Button, Input
from textual.containers import Vertical, Horizontal
from rich.panel import Panel
from rich.align import Align
from datetime import datetime
import os
import shutil
from tkinter import Tk, filedialog

from utils import baca_data, simpan_data

def pilih_file():
    root = Tk()
    root.withdraw()

    file_path = filedialog.askopenfilename(
        title="Pilih Dokumen",
        filetypes=[("PDF files", "*.pdf"), ("Word files", "*.docx")],
    )

    return file_path

class LihatDokumenScreen(Screen):
    def compose(self) -> ComposeResult:
        data = baca_data()

        if not data:
            content = "Tidak ada dokumen."
        else:
            lines = []
            for i, doc in enumerate(reversed(data), start=1):
                lines.append(f'{i}. {doc['nama']}')
                lines.append(f'  File : {doc['file']}')
                lines.append(f'  Tanggal : {doc['tanggal']}')
                lines.append("")
            content = "\n".join(lines)

        yield Header()
        yield Vertical(
            Static(Align.center(Panel(content, title="Daftar Dokumen", expand=False))),
            Horizontal(
                Button("Kembali", id="back"),
                Button("Delete", id="delete", variant="error"),
                Button("Update", id="update", variant="primary"),
            ),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()
        elif event.button.id == "update":
            self.app.push_screen(UpdateScreen())
        elif event.button.id == "delete":

```

```

        self.app.push_screen(DeleteScreen())

class UpdateScreen(Screen):
    def __init__(self):
        super().__init__()
        self.file_path = ""

    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static("Masukkan nomor dokumen yang ingin diupdate"),
            Input(placeholder="Nomor dokumen", id="nomor"),
            Input(placeholder="Nama baru (opsional)", id="nama"),
            Button("Pilih File Baru", id="pilih", variant="primary"),
            Static("", id="file_info"),
            Horizontal(
                Button("Simpan", id="simpan", variant="success"),
                Button("Kembali", id="back"),
            ),
            Static("", id="error"),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()

        elif event.button.id == "pilih":
            file_path = pilih_file()
            info = self.query_one("#file_info", Static)

            if not file_path:
                info.update("Tidak ada file dipilih.")
                return

            if not (file_path.endswith(".pdf") or file_path.endswith(".docx")):
                info.update("Format harus PDF/DOCX.")
                return

            self.file_path = file_path
            info.update(f"File dipilih: {os.path.basename(file_path)}")

        elif event.button.id == "simpan":
            nomor_input = self.query_one("#nomor", Input)
            nama_input = self.query_one("#nama", Input)
            error_text = self.query_one("#error", Static)
            info = self.query_one("#file_info", Static)

            nomor = nomor_input.value.strip()
            nama = nama_input.value.strip()

            if not nomor:
                error_text.update("Nomor harus diisi")
                return

            if not nomor.isdigit():
                error_text.update("Nomor harus berupa angka")
                return

            data = baca_data()
            index = int(nomor)

            if index < 1 or index > len(data):
                error_text.update("Nomor tidak valid")
                return

            if not nama and not self.file_path:
                error_text.update("Isi nama atau pilih file baru")
                return

```

```

real_index = len(data) - index

if nama:
    data[real_index]["nama"] = nama

if self.file_path:
    nama_file = os.path.basename(self.file_path)

    folder_tujuan = "file_dokumen"
    if not os.path.exists(folder_tujuan):
        os.makedirs(folder_tujuan)

    tujuan = os.path.join(folder_tujuan, nama_file)
    shutil.copy(self.file_path, tujuan)

    data[real_index]["file"] = nama_file

data[real_index]["tanggal"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

simpan_data(data)

self.app.pop_screen()
self.app.pop_screen()
self.app.push_screen(LihatDokumenScreen())

class DeleteScreen(Screen):
    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static("Masukkan nomor dokumen yang ingin dihapus"),
            Input(placeholder="Nomor dokumen", id="nomor"),
            Horizontal(
                Button("Hapus", id="hapus", variant="error"),
                Button("Kembali", id="back"),
            ),
            Static("", id="error"),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()

        elif event.button.id == "hapus":
            nomor_input = self.query_one("#nomor", Input)
            error_text = self.query_one("#error", Static)

            nomor = nomor_input.value.strip()

            if not nomor:
                error_text.update("Nomor harus diisi")
                return

            if not nomor.isdigit():
                error_text.update("Nomor harus berupa angka")
                return

            data = baca_data()
            index = int(nomor)

            if index < 1 or index > len(data):
                error_text.update("Nomor tidak valid")
                return

            data.pop(len(data) - index)
            simpan_data(data)

            self.app.pop_screen()
            self.app.pop_screen()
            self.app.push_screen(LihatDokumenScreen())

```

```
def lihat_dokumen(app):
    app.push_screen(LihatDokumenScreen())
```

tambah.py

```
import datetime
import os
import shutil
from tkinter import Tk, filedialog

from textual.app import ComposeResult
from textual.screen import Screen
from textual.widgets import Header, Footer, Static, Button, Input
from textual.containers import Vertical, Horizontal
from rich.panel import Panel
from rich.align import Align

from utils import baca_data, simpan_data

def pilih_file():
    root = Tk()
    root.withdraw()

    file_path = filedialog.askopenfilename(
        title="Pilih Dokumen",
        filetypes=[
            ("PDF files", "*.pdf"),
            ("Word files", "*.docx")
        ]
    )

    return file_path

class TambahDokumenScreen(Screen):
    def __init__(self):
        super().__init__()
        self.file_path = ""

    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(Panel("Tambah Dokumen Baru", expand=False))),
            Input(placeholder="Nama dokumen...", id="nama"),
            Button("Pilih File", id="pilih", variant="primary"),
            Static("", id="file_info"),
            Horizontal(
                Button("Kembali", id="back"),
                Button("Simpan", id="simpan", variant="success"),
                id="action_buttons"
            ),
        ),
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()

        elif event.button.id == "pilih":
            file_path = pilih_file()
            info = self.query_one("#file_info", Static)

            if not file_path:
                info.update("Tidak ada file dipilih.")
            return
```

```

if not (file_path.endswith(".pdf") or file_path.endswith(".docx")):
    info.update("Format harus PDF/DOCX.")
    return

self.file_path = file_path
info.update(f"File dipilih: {os.path.basename(file_path)}")

elif event.button.id == "simpan":
    nama_input = self.query_one("#nama", Input)
    nama = nama_input.value.strip()
    info = self.query_one("#file_info", Static)

    if not nama:
        info.update("Nama dokumen tidak boleh kosong.")
        return

    if not self.file_path:
        info.update("Pilih file terlebih dahulu.")
        return

    data = baca_data()

    for item in data:
        if item["nama"].lower() == nama.lower():
            info.update("Nama dokumen sudah ada.")
            return

    nama_file = os.path.basename(self.file_path)

    folder_tujuan = "file_dokumen"
    if not os.path.exists(folder_tujuan):
        os.makedirs(folder_tujuan)

    tujuan = os.path.join(folder_tujuan, nama_file)

    shutil.copy(self.file_path, tujuan)

    tanggal = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    data.append({
        "nama": nama,
        "file": nama_file,
        "tanggal": tanggal
    })

    simpan_data(data)

    self.app.pop_screen()

def tambah_dokumen(app):
    app.push_screen(TambahDokumenScreen())

```

proses.py

```

from textual.app import ComposeResult
from textual.screen import Screen
from textual.widgets import Header, Footer, Static, Button
from textual.containers import Vertical, Horizontal
from rich.panel import Panel
from rich.align import Align

from utils import baca_data, simpan_data, tambah_rivayat

class ProsesDokumenScreen(Screen):
    def compose(self) -> ComposeResult:

```

```

data = baca_data()

if not data:
    content = "Tidak ada dokumen untuk diproses."
else:
    dokumen = data[-1]
    content = "\n".join([
        "Dokumen teratas akan diproses:",
        "",
        f>Nama : {dokumen['nama']} ",
        f"File : {dokumen['file']} ",
        f"Tanggal : {dokumen['tanggal']} ",
    ])

yield Header()
yield Vertical(
    Static(Align.center(Panel(content, title="Proses Dokumen (LIFO)", expand=False))),

    Horizontal(
        Button("Kembali", id="back"),
        Button("Proses Sekarang", id="proses", variant="primary"),

        id="button-row"
    ),
)

yield Footer()

def on_button_pressed(self, event: Button.Pressed) -> None:
    if event.button.id == "back":
        self.app.pop_screen()

    elif event.button.id == "proses":
        data = baca_data()

        if not data:
            return

        dokumen = data.pop()
        simpan_data(data)
        tambah_rivayat(dokumen)

        result = "\n".join([
            "Dokumen berhasil diproses.",
            "",
            f>Nama : {dokumen['nama']} ",
            f"File : {dokumen['file']} ",
            f"Tanggal : {dokumen['tanggal']} ",
            "",
            "Dokumen telah dipindahkan ke riwayat."
        ])

        self.app.pop_screen()
        self.app.push_screen(HasilProsesScreen(result))

class HasilProsesScreen(Screen):
    def __init__(self, content: str):
        super().__init__()
        self.content = content

    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(Panel(self.content, title="Hasil Proses", expand=False))),
            Horizontal(
                Button("Kembali", id="back"),

                id="button-row"
            ),
        )
        yield Footer()

```

```

def on_button_pressed(self, event: Button.Pressed) -> None:
    if event.button.id == "back":
        self.app.pop_screen()

def proses_dokumen(app):
    app.push_screen(ProsesDokumenScreen())

```

search.py

```

from textual.app import ComposeResult
from textual.screen import Screen
from textual.widgets import Header, Footer, Static, Button, Input
from textual.containers import Vertical, Horizontal
from rich.panel import Panel
from rich.align import Align

from utils import baca_data

class SearchDokumenScreen(Screen):
    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(Panel("Masukkan nama dokumen untuk mencari", title="Search Dokumen", expand=False))),
            Input(placeholder="Nama dokumen...", id="keyword"),
            Horizontal(
                Button("Kembali", id="back"),
                Button("Cari", id="cari", variant="primary"),
                id="buttons"
            ),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()

        elif event.button.id == "cari":
            input_widget = self.query_one("#keyword", Input)
            keyword = input_widget.value.lower().strip()

            if not keyword:
                return

            data = baca_data()

            if not data:
                result = "Tidak ada data dokumen."
            else:
                hasil = []
                for d in data:
                    if keyword in d["nama"].lower():
                        hasil.append(d)

            if not hasil:
                result = "Tidak ditemukan dokumen yang cocok."
            else:
                lines = [f"Ditemukan {len(hasil)} dokumen", ""]
                for i, d in enumerate(hasil, 1):
                    lines.append(f"{i}. {d['nama']}")
                    lines.append(f"   File   : {d['file']}")
                    lines.append(f"   Tanggal : {d['tanggal']}")
                    lines.append("")
                result = "\n".join(lines)

```



```

        self.app.push_screen(HasilSearchScreen(result))

class HasilSearchScreen(Screen):
    def __init__(self, content: str):
        super().__init__()
        self.content = content

    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(Panel(self.content, title="Hasil Pencarian", expand=False))),
            Input(placeholder="Cari lagi...", id="keyword"),
            Horizontal(
                Button("Kembali", id="back"),
                Button("Cari", id="cari", variant="primary"),
                id="buttons"
            ),
        )
        yield Footer()

    def on_button_pressed(self, event: Button.Pressed) -> None:
        if event.button.id == "back":
            self.app.pop_screen()

        elif event.button.id == "cari":
            input_widget = self.query_one("#keyword", Input)
            keyword = input_widget.value.lower().strip()

            if not keyword:
                return

            data = baca_data()

            if not data:
                result = "Tidak ada data dokumen."
            else:
                hasil = []
                for d in data:
                    if keyword in d["nama"].lower():
                        hasil.append(d)

                if not hasil:
                    result = "Tidak ditemukan dokumen yang cocok."
                else:
                    lines = [f"Ditemukan {len(hasil)} dokumen", ""]
                    for i, d in enumerate(hasil, 1):
                        lines.append(f"{i}. {d['nama']}")
                        lines.append(f"   File   : {d['file']}")
                        lines.append(f"   Tanggal : {d['tanggal']}")
                        lines.append("")
                    result = "\n".join(lines)

            self.app.push_screen(HasilSearchScreen(result))

def search_dokumen(app):
    app.push_screen(SearchDokumenScreen())

```

sort.py

```

from textual.app import ComposeResult
from textual.screen import Screen
from textual.widgets import Header, Footer, Static, Button
from textual.containers import Vertical, Horizontal
from rich.panel import Panel
from rich.align import Align

```

```

from utils import baca_data

def merge_sort(data, key):
    if len(data) <= 1:
        return data

    mid = len(data) // 2
    left = merge_sort(data[:mid], key)
    right = merge_sort(data[mid:], key)

    return merge(left, right, key)

def merge(left, right, key):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key].lower() <= right[j][key].lower():
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort_tanggal(data):
    if len(data) <= 1:
        return data

    mid = len(data) // 2
    left = merge_sort_tanggal(data[:mid])
    right = merge_sort_tanggal(data[mid:])

    return merge_tanggal(left, right)

def merge_tanggal(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i]["tanggal"] <= right[j]["tanggal"]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

class SortDokumenScreen(Screen):
    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(
                Panel("Pilih metode sorting", title="Sorting Dokumen", expand=False)
            )),
            Horizontal(
                Button("Nama (A-Z)", id="nama", variant="primary"),
                Button("Tanggal", id="tanggal", variant="primary"),
            ),
            Horizontal(
                Button("Kembali", id="back"),
            )
        )

```

```

    )
    yield Footer()

def on_button_pressed(self, event: Button.Pressed) -> None:
    if event.button.id == "back":
        self.app.pop_screen()

    elif event.button.id == "nama":
        data = baca_data()
        if not data:
            result = "Tidak ada data dokumen."
        else:
            hasil = merge_sort(data, "nama")
            result = self.format_hasil(hasil, "Nama (A-Z)")
            self.app.push_screen(HasilSortScreen(result))

    elif event.button.id == "tanggal":
        data = baca_data()
        if not data:
            result = "Tidak ada data dokumen."
        else:
            hasil = merge_sort_tanggal(data)
            result = self.format_hasil(hasil, "Tanggal (Terlama → Terbaru)")
            self.app.push_screen(HasilSortScreen(result))

def format_hasil(self, data, metode):
    lines = [f"Hasil Sorting: {metode}", ""]
    for i, d in enumerate(data, 1):
        lines.append(f"{i}. {d['nama']}")
        lines.append(f"   File   : {d['file']}")
        lines.append(f"   Tanggal : {d['tanggal']}")
        lines.append("")
    return "\n".join(lines)

class HasilSortScreen(Screen):
    def __init__(self, content: str):
        super().__init__()
        self.content = content

    def compose(self) -> ComposeResult:
        yield Header()
        yield Vertical(
            Static(Align.center(
                Panel(self.content, title="Hasil Sorting", expand=False)
            )),
            Horizontal(
                Button("Kembali", id="back"),
            )
        )
        yield Footer()

def on_button_pressed(self, event: Button.Pressed) -> None:
    if event.button.id == "back":
        self.app.pop_screen()

def sort_dokumen(app):
    app.push_screen(SortDokumenScreen())

```

